Object-Oriented Programming Using Java

I/O Streams

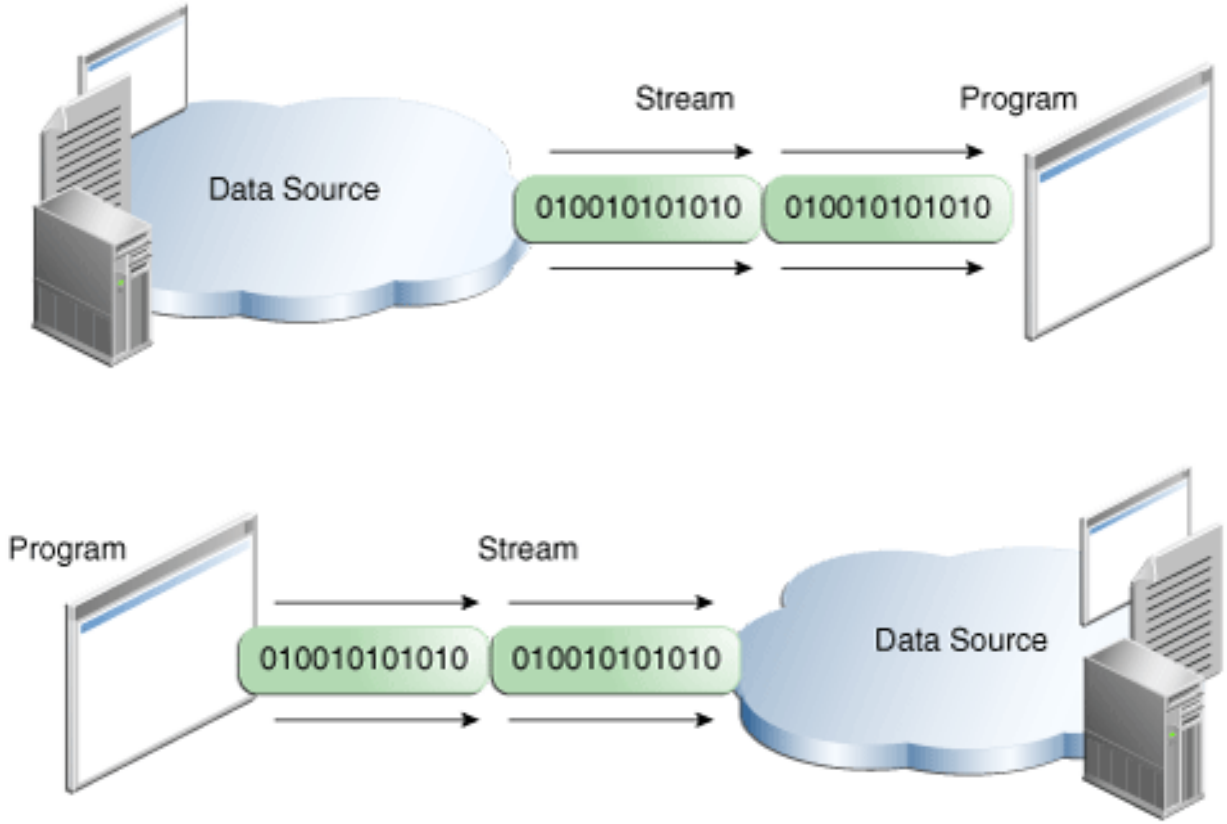
Contents

- How to access text files.
- How to read/write objects from/to files.

References:
https://docs.oracle.com/javase/tutorial/essential/io/streams.html

What are streams?

- Reading/Writing information into/from a program.

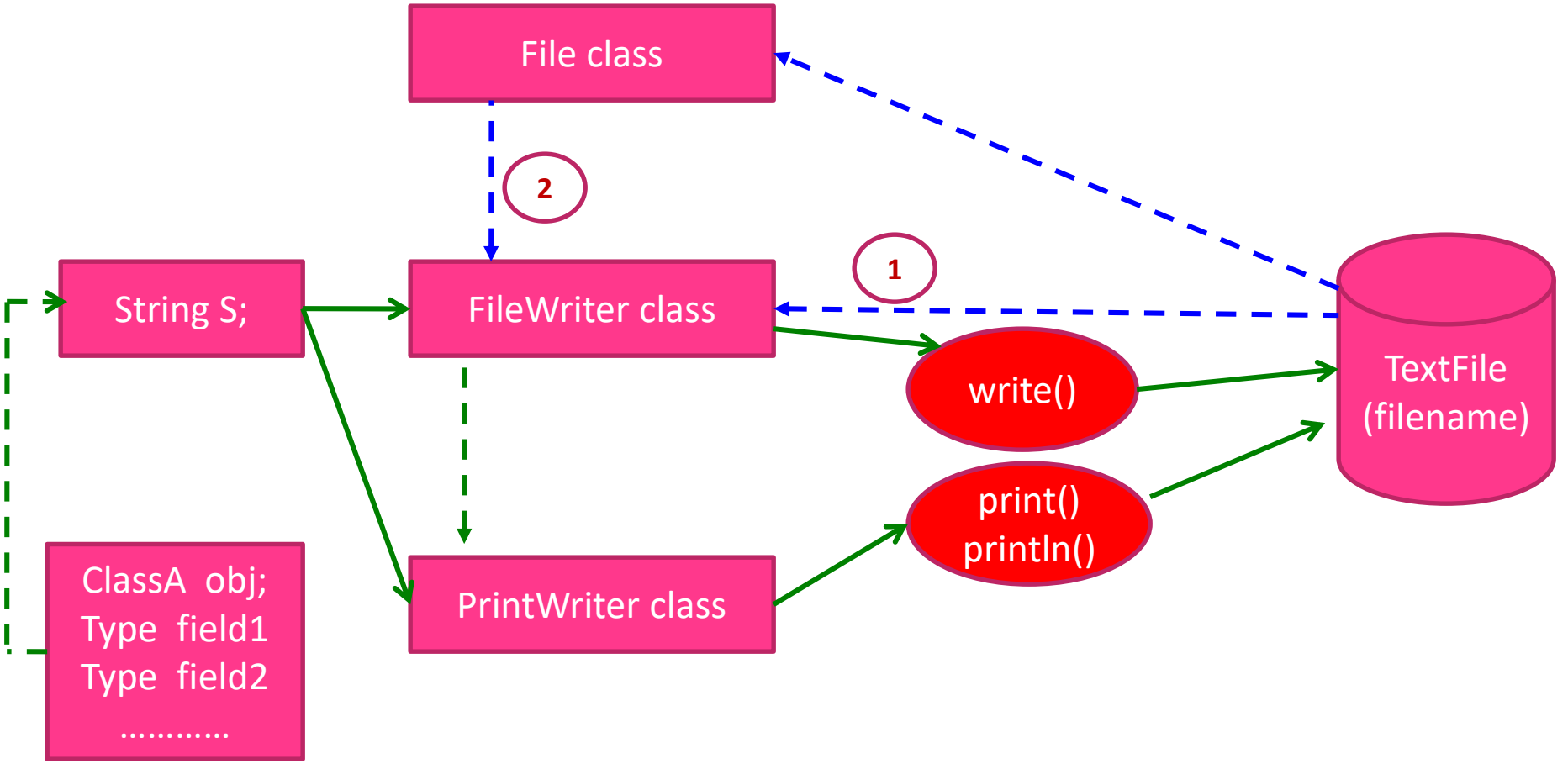


Access Text Files

- java.io.**Reader** (implements java.io.Closeable, java.lang.Readable) (abstract)
  - java.io.**BufferedReader**
    - java.io.**LineNumberReader**
  - java.io.**CharArrayReader**
  - java.io.**FilterReader**
    - java.io.**PushbackReader**
  - java.io.**InputStreamReader**
    - java.io.**FileReader**
  - java.io.**PipedReader**
  - java.io.**StringReader**

- java.io.**Writer** (implements java.lang.Appendable, java.io.Closeable, java.io.Flushable) (abstract)
  - java.io.**BufferedWriter**
  - java.io.**CharArrayWriter**
  - java.io.**FilterWriter**
  - java.io.**OutputStreamWriter**
    - java.io.**FileWriter**
  - java.io.**PipedWriter**
  - java.io.**PrintWriter**
  - java.io.**StringWriter**

Writing Text Files

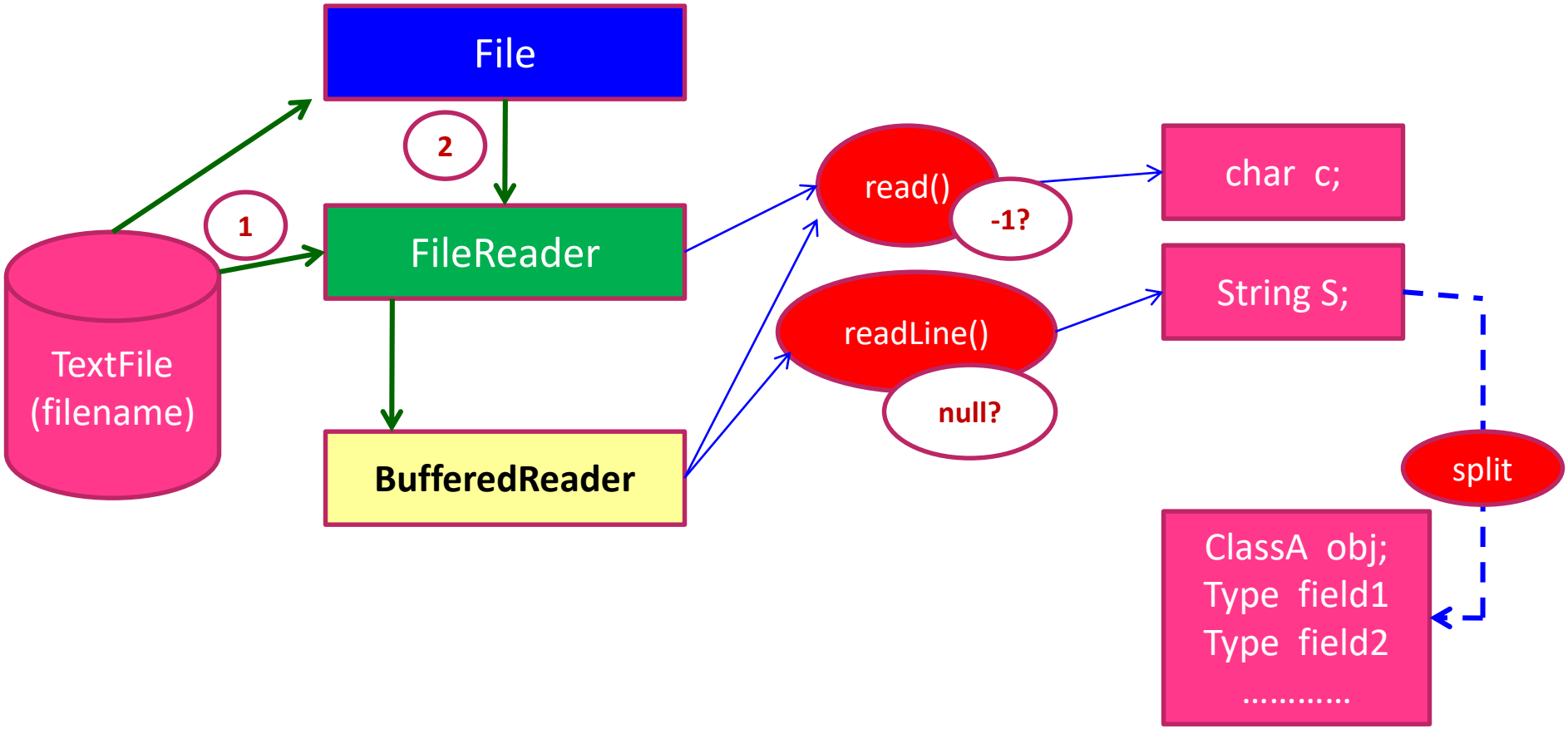


VD1: Ghi 1 dòng xuống file

```
try
{
    File f = new File("Test.txt");
    FileWriter fw = new FileWriter(f);
    fw.write("Line 1.1 ");
    fw.write("Line 1.2 ");
    fw.write("Line 1.3 ");

    PrintWriter pw = new PrintWriter(fw);
    pw.println();//new line
    pw.println("Line 2");
    pw.println("Line 3");
    pw.println("Line 4");

    fw.close(); pw.close();
    System.out.println("Save to file successful");
}
catch (Exception e)
{
    System.out.println("Failed: " + e.getMessage());
}
```

```
boolean append = true;
File f = new File("Test.txt");
FileWriter fw = new FileWriter(f, append);
PrintWriter pw = new PrintWriter(fw);
pw.println("Line 5 append");
pw.println("Line 6 append");

fw.close(); pw.close();
```

Reading Text Files

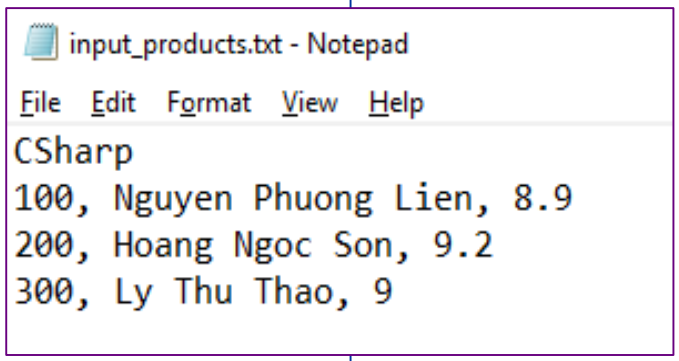


VD2: Đọc toàn bộ file

```
try
{
    File f = new File("Test.txt");
    FileReader fr = new FileReader(f);
    BufferedReader br = new BufferedReader(fr);

    // read 1 line
    String line = br.readLine();
    while (line != null){
        System.out.println(line);
        line = br.readLine();
    }

    fr.close(); br.close();
}
catch (Exception e)
{
    System.out.println("Failed: " + e.getMessage());
}
```

VD3: Ghi danh sách xuống file

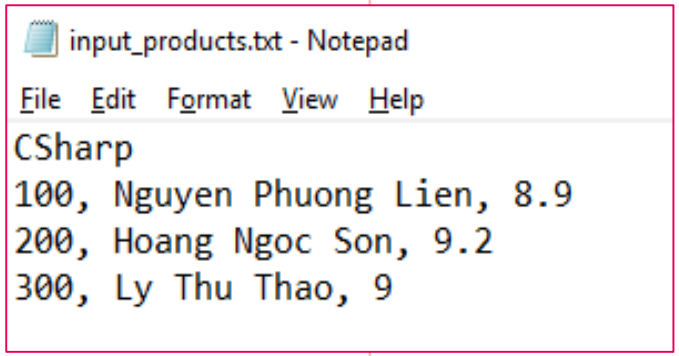
```java
// Ghi nội dung của list xuống file
public static void saveToFile(String fileName, ArrayList<Student> list) {
    File f = new File(fileName);
    try{
        FileWriter fw = new FileWriter(f);
        PrintWriter pw = new PrintWriter(fw);
        pw.println(Student.course);
        for (Student stud:list){
            pw.println(stud);
        }
        pw.close();
        fw.close();
    }
    catch (Exception e){
        System.out.println(e);
    }
}
```




```java
ArrayList<Student> list = new ArrayList<Student>();
list.add(new Student(100, "Nguyen Cat Hanh", 12));
list.add(new Student(200, "Le Cat Hanh", 9.1f));
list.add(new Student(300, "Ngo Cat Hanh", 9.1f));
list.add(new Student(400, "Phan Cat Hanh", 4.5f));

Student.course = "CSharp";
Lib.saveToFile("students.txt", list);
```

VD4: Đọc 1 dòng từ file và chuyển thành đối tượng

```
public void addFromFile(String fileName) {
    File f = new File(fileName);
    try{
        FileReader fr = new FileReader(f);
        BufferedReader br = new BufferedReader(fr);
        String course, line;
        //Đọc dòng đầu tiên (course)
        if ((course = br.readLine())!=null){
            Instructor.course = course.trim();
        }
        //Đọc các dòng còn lại
        while ((line = br.readLine())!=null){
            if (line.trim().equals("")) continue;
            String arr[] = line.split("[,]+");
            Instructor ins = new Instructor(arr[0], arr[1], Float.parseFloat(arr[2].trim()));
            list.add(ins);
        }
        br.close();
        fr.close();
    }
    catch (Exception e){
        System.out.println(e);
    }
}
```



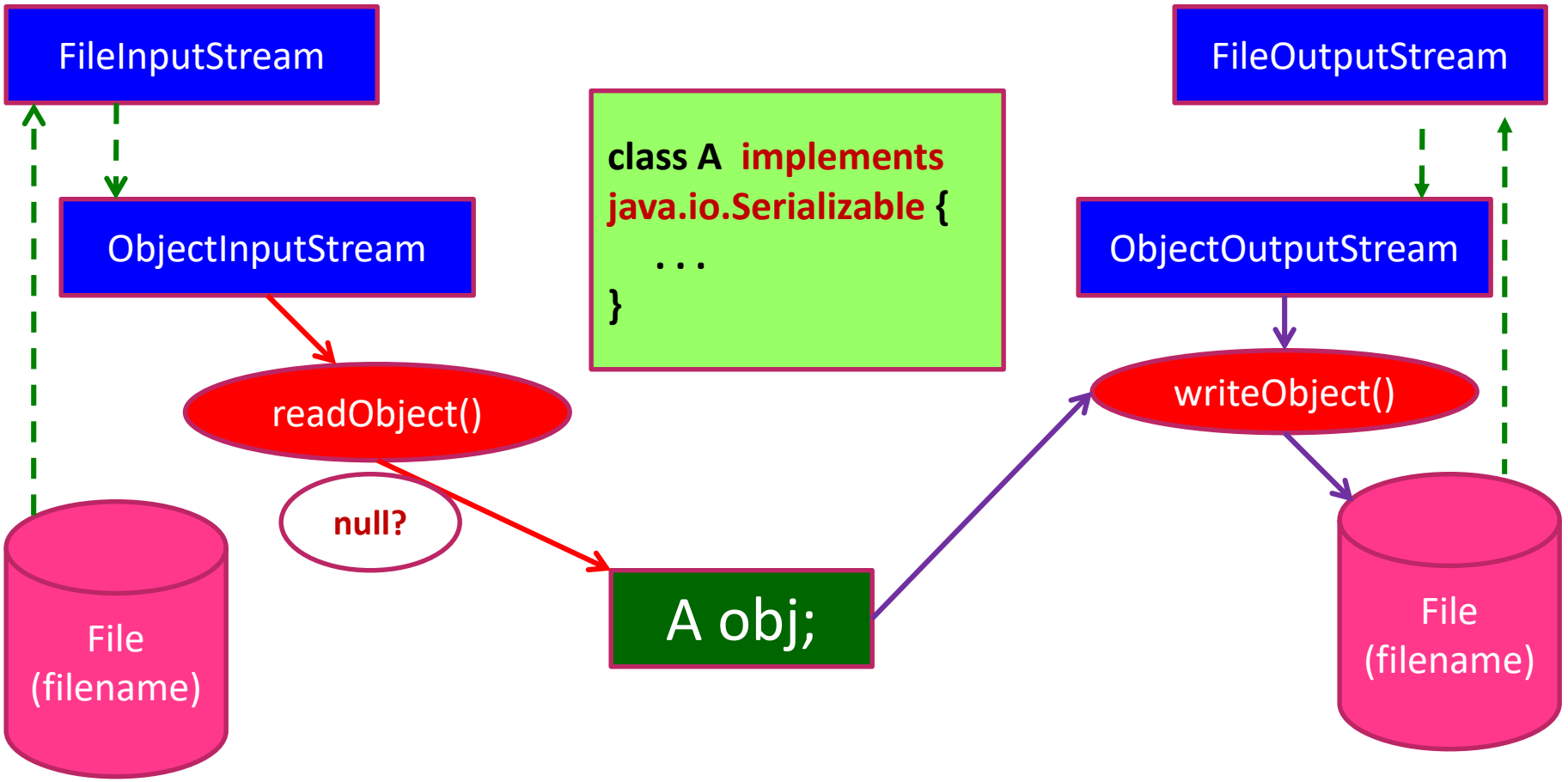
Access Object Files

- java.lang.**Object**
  - java.io.**InputStream** (implements java.io.Closeable)
    - java.io.**ByteArrayInputStream**
    - java.io.**FileInputStream**
    - java.io.**FilterInputStream**
    - java.io.**ObjectInputStream** (implements java.io.ObjectInput, java.io.ObjectStreamConstants)
  - java.io.**OutputStream** (implements java.io.Closeable, java.io.Flushable)
    - java.io.**ByteArrayOutputStream**
    - java.io.**FileOutputStream**
    - java.io.**FilterOutputStream**
    - java.io.**ObjectOutputStream** (implements java.io.ObjectOutput, java.io.ObjectStreamConstants)

Serialization and De-serialization

- **Serialization** is a task which will concate all data of an object to a byte stream then it can be written to a data source.
- Static and transient data can not be serialized.
- **De-serialization** is a task which will read a byte stream from a data source , split the stream to fields then assign them to data fields of an object appropriately.

How to read/write objects from/to files



Writing Objects to files

```java
class Student implements Serializable{
    String id, name;

    public Student(String id, String name) {
        this.id = id;
        this.name = name;
    }
    @Override
    public String toString() {
        return id + " - " + name;
    }
}
```

```java
try
{
    File f = new File("Test.dat");
    FileOutputStream fos = new FileOutputStream(f);
    ObjectOutputStream ous = new ObjectOutputStream(fos);
    ous.writeObject(new Student("id01", "name01"));
    ous.writeObject(new Student("id02", "name02"));
    ous.writeObject(new Student("id03", "name03"));

    fos.close(); ous.close();
    System.out.println("Done");
}
catch (Exception e)
{
    System.out.println("Failed: " + e.getMessage());
}
```

Reading Objects from files

```
try
{
    File f = new File("Test.dat");
    FileInputStream fis = new FileInputStream(f);
    ObjectInputStream ois = new ObjectInputStream(fis);

    Student stud = (Student)ois.readObject();
    while (stud!=null){
        System.out.println(stud);
        stud = (Student)ois.readObject();
    }
    fis.close(); ois.close();
}
catch (EOFException e){
    //nothing to do
}
catch (Exception e)
{
    System.out.println("Failed: " + e.getMessage());
}
```

Assignment

Xây dựng chương trình quản lý sinh viên, dữ liệu được lưu vào file .txt, bao gồm các chức năng

1. Thêm sinh viên
2. Xóa sinh viên
3. Sửa thông tin sinh viên
4. Xem danh sách sinh viên
5. Sắp xếp danh sách sinh viên theo tên
6. Tìm sinh viên có ĐTB cao nhất